



Nombre:

Grupo:

Pregunta 1: Contestar V/F las siguientes cuestiones, teniendo en cuenta que una respuesta incorrecta invalida una correcta.

[] La siguiente definición de vector estático necesita de constructor copia y operador de asignación:

```
template<typename T> class MiVect{
    int tama;
    T *v;
public:
    MiVect(int n){ v=new T[tama=n]; }
    ...
};
```

[] La misma clase anterior debe definir así el operador corchete “[]” para que funcione correctamente (se obvian las comprobaciones de rango):

```
T operator[](unsigned i) { return v[i]; }
```

[] Si se ha instanciado en la clase *Biblioteca* a *Mivect* para implementar una relación de asociación con la clase *Libro* como:

```
MiVect<Libro*> estante;
```

Entonces el destructor de *Biblioteca* debe entonces eliminar estante ejecutando:

```
delete[] estante;
```

[] Esta sería una posible instrucción dentro de la función que elimina el nodo apuntado por el puntero *p* de una lista doblemente enlazada:

```
p->sig->ant = p->ant;
```

[] Un vector de listas es un contenedor válido en términos de eficiencia para implementar una cola con prioridad y un grafo aunque no tanto para una matriz dispersa

[] En un árbol AVL, tanto el borrado como la inserción requieren la localización de algún nodo hoja durante el proceso.

[] La unión de dos conjuntos disjuntos de tamaños *n* y *m* puede llevarse a cabo mediante una operación en $O(1)$

[] Un árbol B de orden cinco que está indexando 500 datos puede tener altura 3 (4 niveles)

[] Para evitar tanto agrupamientos primarios como secundarios a la hora de hacer hashing es preferible utilizar dispersión cuadrática que dispersión doble.

[] Estas dos definiciones son válidas para almacenar eficientemente las reservas de un hotel por fecha de emisión:

```
map<Fecha, list<Reserva> >
multimap <Fecha, Reserva>
```

Pregunta 2:

Mostrar la evolución de un heap tras la realización de las siguientes operaciones (mostrar sólo la representación compacta en el vector de soporte): push(8), push(7), push(1), push(2), push(9), push(5), pop(), pop(), push(3). Menor valor numérico implica mayor prioridad.

Pregunta 3:

(a) Un árbol ABB puede ser un árbol perfecto. Escribid el código de la siguiente función que determina si un árbol ABB es o no un árbol perfecto a una profundidad *h*, es decir, si independientemente de su altura, tiene todos los niveles completos hasta la profundidad *h*.

```
template<typename T>
bool ABB<T>::arbolPerfecto(unsigned h);
```

(b) Implementar la función de búsqueda utilizada por la central nacional de correos y que establece una relación entre cualquier código postal del destinatario y la central de distribución de correos que le corresponde.

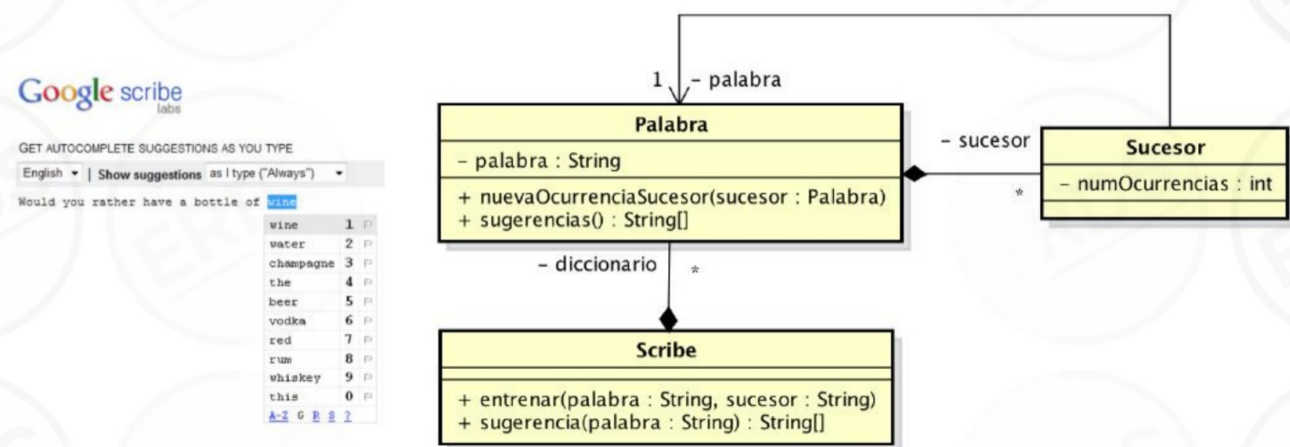
```
enum estado {vacio, dato, borrado};
struct Entrada{
    unsigned cp; //código postal
    Central* cc; //central de distribución de correos
    estado est;
};
class THashCorreos{
    vector<Entrada> codigos;
    unsigned tamaTabla;
public:
    ...
```

```
    unsigned hash(unsigned codpost, unsigned intento);
    bool buscar(unsigned codpost, Central*& c);
};
```

Pregunta 4:

Uno de los proyectos más interesantes de Google Labs fue **Google Scribe** (cancelado hace unos años), que consistía en un editor de textos que ofrecía sugerencias de la palabra a continuación conforme se escribía. La clave es diseñar una estructura de datos que guarde por cada palabra la lista de palabras que pueden aparecer a continuación con mayor probabilidad. Para que sea efectivo este sistema debe entrenarse con una gran cantidad de textos variados de forma que se construyan estas listas de sucesores.

Vamos a implementar una versión muy sencilla de Google Scribe, ilustrado por el diagrama UML mostrado a continuación.



La clase Scribe, que representa a nuestro generador de sugerencias, contiene una estructura de datos con la lista de palabras reconocidas (*diccionario*). Cada una de las palabras contiene a su vez las palabras sucesoras (sucesor), con el número total de ocurrencias de cada una encontrados durante el entrenamiento (e.g. of->wine (1024), water (732), champagne (239)...).

La operación *entrenar()* pasa una palabra y un sucesor de ésta al sistema. Ésta crea una entrada en el diccionario para la palabra y su sucesor si alguno de ellos no existe. A continuación llama a la operación *nuevaOcurrenciaSucesor()* de la palabra, que a su vez crea la entrada para el sucesor si no existe o incrementa su número de ocurrencias en caso contrario. La operación *sugerencia()* consulta la estructura de datos para devolver los 10 sucesores más probables (i.e. con mayor número de ocurrencias) para la palabra dada.

Implementar las clases del diagrama y además realizar un pequeño programa *main()* que cree una instancia de Scribe y la entrene con el contenido de un fichero de texto “texto-largo.txt”. Para ello deberá ir leyendo caracteres y construyendo palabras. Cada vez que tenga una palabra nueva *pn* llamará a *entrenar(pa, pn)* siendo *pa* la palabra anterior identificada en el texto. Además de caracteres alfanuméricos, en el texto pueden aparecer los caracteres de puntuación punto, coma, punto y coma, dos puntos, símbolos de interrogación y admiración. Una palabra termina al encontrar un espacio o un caracter de puntuación, pero nunca se considerará sucesora a una palabra que sigue a otra habiendo un carácter de puntuación enmedio. Por ejemplo, en el texto “Y de pronto pensé ¿Qué hago aquí?” “pensé” es sucesora de “pronto”, pero “qué” no lo es de “pensé”.